¿Cambió el objeto o cambió la base de datos?

Enunciado

En el sistema Héroes, implementar una pantalla de laboratorio que permita consultar un héroe y modificar su nombre. El usuario elegirá si desea guardar o únicamente probar el cambio en memoria. Comparar los resultados de ambas operaciones y explicar el papel del contexto.

Requisitos al comenzar

- Proyecto MVC que compile y conexión funcional.
- Contexto y entidad preparados por el docente.
- Al menos un héroe registrado y conocimiento de su identificador.
- Acceso a SSMS para comprobar el dato persistido.

Paso 1. Reconocer los componentes

Abrir `Heroe.cs`, `HeroesDbContext.cs` y `Program.cs`. Identificar la clave, el conjunto de entidades y el registro de inyección.

Completar:

- Clase de entidad: __.
- Clave primaria: __.
- Clase de contexto: __.
- Propiedad de acceso al conjunto: __.
- Lugar donde se configura el proveedor y la conexión: __.

Paso 2. Agregar el controlador

En Visual Studio, clic derecho en **Controllers** → **Agregar** → **Controlador**. Seleccionar **Controlador MVC: vacío** y nombrarlo `LaboratorioEfController`. Según la versión de Visual Studio, esa opción puede estar dentro de **Nuevo elemento con scaffolding**. Elegir la opción vacía, pues el CRUD generado se trabajará en la clase 4.

Reemplazar su contenido por:

```
using HeroesMvc.Data;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
```

```
namespace HeroesMvc.Controllers;

public class LaboratorioEfController : Controller
{
    private readonly HeroesDbContext _context;

    public LaboratorioEfController(HeroesDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public async Task<IActionResult> Editar(int id)
    {
        var heroe = await _context.Heroes
            .FirstOrDefaultAsync(h => h.Id == id);

        if (heroe is null)
            return NotFound();

        return View(heroe);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Editar(
        int id, string? nombre, bool guardar = false)
    {
        var heroe = await _context.Heroes
            .FirstOrDefaultAsync(h => h.Id == id);

        if (heroe is null)
            return NotFound();

        if (string.IsNullOrWhiteSpace(nombre) || nombre.Trim().Length > 100)
        {
            ModelState.AddModelError(
                "Nombre", "Ingrese un nombre de 1 a 100 caracteres.");
            return View(heroe);
        }

        var nombreOriginal = heroe.Nombre;
        heroe.Nombre = nombre.Trim();

        // Se explicita para observar el estado durante el laboratorio.
        _context.ChangeTracker.DetectChanges();
```

```
        var estadoAntes = _context.Entry(heroe).State;

        if (guardar)
        {
            await _context.SaveChangesAsync();

            TempData["Resultado"] =
                $"Antes: {estadoAntes}. Después:
{_context.Entry(heroe).State}. " +
                "Se ejecutó SaveChangesAsync; compruebe el dato en SQL
Server.";

            return RedirectToAction(nameof(Editar), new { id });
        }

        ViewData["Resultado"] =
            $"Original: {nombreOriginal}. En memoria: {heroe.Nombre}. " +
            $"Estado: {estadoAntes}. No se llamó a SaveChangesAsync.";

        return View(heroe);
    }
}
```

Lectura guiada: el constructor recibe el contexto. GET consulta para mostrar la pantalla. POST vuelve a consultar, cambia una propiedad y decide si guardar. El navegador no conserva el contexto entre GET y POST.

El POST utiliza parámetros simples para concentrar la práctica en persistencia. ViewModel y validación completa se profundizarán en las clases 6 y 7. La comprobación mínima evita enviar un nombre vacío o mayor que la columna.

Paso 3. Agregar la vista

1. Dentro de **Views**, agregar la carpeta `LaboratorioEf`.
2. Clic derecho en esa carpeta → **Agregar** → **Vista** o **Nuevo elemento** → **Vista Razor vacía**, según la versión de Visual Studio.
3. Crear `Editar.cshtml`, sin `PageModel` ni directiva `@page`, porque es una vista MVC.
4. Confirmar que `Views/_ViewImports.cshtml` contiene:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Escribir en `Editar.cshtml`:

```
@model HeroesMvc.Models.Heroe

@{
    ViewData["Title"] = "Laboratorio EF Core";
    var resultado = TempData["Resultado"] as string
        ?? ViewData["Resultado"] as string;
}

<h1>Laboratorio EF Core</h1>
<p>Héroe consultado: @Model.Id</p>

@if (resultado is not null)
{
    <div class="alert alert-info">@resultado</div>
}

<form asp-controller="LaboratorioEf" asp-action="Editar" method="post">
    <input type="hidden" asp-for="Id" />

    <div class="mb-3">
        <label asp-for="Nombre" class="form-label"></label>
        <input asp-for="Nombre" class="form-control"
               maxlength="100" required />
        <span asp-validation-for="Nombre" class="text-danger"></span>
    </div>

    <div class="form-check mb-3">
        <input type="checkbox" id="guardar" name="guardar"
               value="true" class="form-check-input" />
        <label for="guardar" class="form-check-label">
            Guardar el cambio en la base de datos
        </label>
    </div>

    <button type="submit" class="btn btn-primary">Ejecutar prueba</button>
    <a asp-action="Editar" asp-route-id="@Model.Id"
       class="btn btn-secondary">Consultar nuevamente</a>
</form>
```

El Form Tag Helper genera el token antifalsificación para este formulario POST. El controlador lo valida. Una casilla desmarcada no envía `guardar`; el parámetro toma su valor predeterminado `false`.

Paso 4. Ejecutar las pruebas

Ejecutar la aplicación y navegar a `/LaboratorioEf/Editar/1`, sustituyendo `1` por el identificador real.

En SSMS usar:

```
USE HeroesClase1;
SELECT Id, Nombre FROM dbo.Heroes WHERE Id = 1;
```

Prueba A: modificar sin guardar

1. Anotar el nombre original en SSMS.
2. Escribir un nombre diferente, por ejemplo `Centinela Azul`.
3. Dejar la casilla de guardar desmarcada y enviar.
4. Observar el mensaje y el nombre del formulario.
5. Ejecutar de nuevo el SELECT en SSMS.
6. Pulsar **Consultar nuevamente**.

Resultado esperado: se muestra el nuevo nombre en la respuesta del POST, pero SQL conserva el original. La nueva consulta recupera el original. Con un valor diferente, el estado detectado antes del guardado es `Modified`.

Prueba B: modificar y guardar

1. Escribir nuevamente un nombre diferente del persistido.
2. Activar la casilla y enviar.
3. Observar el mensaje tras la redirección.
4. Repetir el SELECT y consultar nuevamente.

Resultado esperado: SQL y la nueva consulta muestran el nuevo nombre. El estado antes del guardado es `Modified` y después de un guardado correcto es `Unchanged`.

Prueba C: identificador inexistente

Abrir `/LaboratorioEf/Editar/999999`, verificando que ese identificador no exista.

Resultado esperado: respuesta HTTP 404 por `NotFound()`.

Caso para discutir

Enviar exactamente el nombre ya almacenado. Con esta entidad y este flujo, no se detecta una modificación de `Nombre`; el estado permanece `Unchanged`. Por eso las pruebas A y B exigen utilizar un valor diferente.

Paso 5. Evidencias y preguntas

Entregar:

1. Código del controlador y de la vista.
2. Captura del resultado sin guardar y SELECT donde siga el valor original.
3. Captura del resultado guardado y SELECT con el valor nuevo.
4. Respuestas breves a las preguntas siguientes.

Preguntas de comprensión

1. ¿Qué diferencia existe entre `Heroe`, `DbSet<Heroe>` y `HeroesDbContext`?
2. ¿Por qué el formulario puede mostrar un nombre nuevo aunque SQL conserve el anterior?
3. ¿Qué hace `SaveChangesAsync()` en esta práctica?
4. ¿Por qué el POST vuelve a consultar el héroe?
5. ¿Por qué no necesitamos llamar a `Update(heroe)`?
6. ¿Para qué se utilizó `DetectChanges()`?
   Deben subir txt o MD con las respuestas de las preguntas.

Errores frecuentes y solución

Síntoma	Qué revisar
No se resuelve `HeroesDbContext` al crear el controlador	Registro `AddDbContext`, namespaces y coincidencia del tipo registrado con el solicitado
Error de conexión	Instancia SQL, base, credenciales y cadena efectiva utilizada
`Invalid object name 'Heroes'`	Base conectada, tabla y esquema del mapeo
Vista no encontrada	`Views/LaboratorioEf/Editar.cshtml`
Los atributos `asp-` no funcionan	Registro de Tag Helpers en `Views/_ViewImports.cshtml`
404 al consultar	Identificador existente y ruta convencional MVC
Estado `Unchanged` en la prueba	Usar un nombre distinto y ejecutar DetectChanges antes de inspeccionar
SQL no cambia aun marcando guardar	Breakpoint en POST, valor `guardar`, ejecución de SaveChangesAsync y base consultada en SSMS
Nombre nuevo aparece una vez y luego desaparece	Comportamiento esperado de la prueba sin guardar

Fuentes técnicas

Documentación oficial utilizada para revisar los conceptos:

- Descripción general de EF Core
- Seguimiento de cambios
- Detección de cambios
- Guardado de datos
- Configuración y duración de DbContext